

汽车销售管理系统（SSM + Vue3）

day01- 新主题选题、环境搭建与Git入门-开发文档

预期成果：

- 完成分组，确定新主题项目方向
- 完成开发环境安装与验证（JDK、MySQL、Maven、Node.js、Git、Tomcat）
- 掌握Git基本命令，成功创建仓库并推送代码
- 后端SSM脚手架导入IDEA并运行成功（数据库连接测试通过）
- 前端Vue3+Vite项目创建并启动成功（`npm run dev` 可访问）
- 前后端项目代码已提交至Git仓库，并推送到远程仓库（含 `main` 和 `dev` 分支）
- 初步完成新主题项目的表结构设计（6~8张表）

一、实训简介

1.1 实训定位

本次实训采用“案例教学 + 项目实战”双轨模式。以 **汽车销售管理系统** 为教学载体，带领大家完整走一遍全栈开发流程；各组需**迁移至新主题项目**，将所学技能独立应用到新场景中。

1.2 教学载体：汽车销售管理系统

本系统是一个完整的全栈应用，包含以下核心模块：

模块	功能说明
车辆管理	品牌维护、车辆增删改查、分页搜索、库存管理
试驾预约	客户提交预约、时间冲突校验、管理员审核确认
销售订单	下单扣库存、事务控制
销量统计	按品牌统计订单量，ECharts图表展示
用户认证	三表分离登录（user + employee + customer）、JWT权限控制

技术栈：

- 后端：Spring + SpringMVC + MyBatis (SSM) + MySQL 8.0 + Maven
- 前端：Vue 3 + Vite + Element Plus + Axios + ECharts
- 版本控制：Git

团队分工（每组3~5人）：

- 后端（1~2人）：车辆、品牌、客户、试驾预约、订单、库存、统计模块
- 前端（1~2人）：车辆列表、订单页面、统计图表、试驾预约表单、布局、个人中心
- 测试/文档（1人）：测试用例、Bug记录、答辩PPT

1.3 新主题项目（实训核心）

在掌握汽车销售系统的基础上，**每组选择一个新主题**进行开发，将所学技术迁移至真实业务场景。

为什么这样做？

- 避免所有小组项目雷同，增加答辩多样性
- 培养“举一反三”的能力——学会将一个业务模型迁移到另一个业务
- 更接近真实工作场景：根据需求文档，从零开始设计

主题范围（仅供参考，不限定）：

主题	核心业务	对标汽车销售系统的模块映射
图书管理系统	图书借阅、读者管理、库存管理	车辆→图书，客户→读者，试驾预约→借阅预约
健身房会员管理系统	会员卡管理、课程预约、签到	车辆→课程，客户→会员，试驾预约→课程预约
活动报名系统	活动发布、报名审核、签到	车辆→活动，客户→报名人，试驾预约→报名申请
宠物领养系统	宠物管理、领养申请、审核	车辆→宠物，客户→领养人，试驾预约→领养申请
二手交易平台	商品发布、订单交易、评价	车辆→商品，销售订单→交易订单，试驾预约→面交预约
会议室预约系统	会议室管理、预约、审核	车辆→会议室，客户→员工，试驾预约→会议预约

说明：第一个案例手把手带领大家完成，第二个主题由各组自主完成，以此检验学习效果。

1.4 举例：从汽车销售系统迁移至新主题

汽车销售系统作为教学参考案例，其代码结构完全适用于其他业务主题。迁移时只需调整以下内容：

迁移项	原案例（汽车销售）	新主题（以“图书管理”为例）	迁移方式
表结构	brand, car, customer...	book, category, member...	按业务重新设计，6~8张表
实体类	Brand, Car, Customer...	Book, Category, Member...	替换表名，字段对应修改
Mapper	BrandMapper, CarMapper...	BookMapper, CategoryMapper...	重新生成逆向工程
Service	CarService, OrderService...	BookService, BorrowService...	修改业务逻辑，复用事务模式
Controller	CarController, OrderController...	BookController, BorrowController...	修改接口路径，复用返回格式
前端页面	车辆列表、预约表单、订单列表	图书列表、借阅表单、借阅记录	复用布局和组件，替换字段名
业务校验	车型重复校验、预约冲突校验	图书编号唯一校验、借阅冲突校验	复用校验逻辑

各小组在第一天需完成任务：

1. **确定主题**：组长组织讨论，选择本组的项目主题
2. **需求分析**：参照汽车销售系统的业务流程图，绘制本主题的业务流程图
3. **初步设计**：列出核心功能模块（对标汽车销售系统的5大模块）
4. **表结构规划**：思考本主题需要几张表（建议6~8张）

二、开发环境安装

2.1 JDK 1.8 (Java运行环境)

1. 下载安装包：<https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>（选择Windows x64）
2. 安装时记住安装路径，例如 `C:\Program Files\Java\jdk1.8.0_202`
3. 配置环境变量：
 - 右键“此电脑”→属性→高级系统设置→环境变量
 - 新建系统变量：变量名 `JAVA_HOME`，变量值 `C:\Program Files\Java\jdk1.8.0_202`
 - 找到 `Path` 变量，添加 `%JAVA_HOME%\bin`
4. 验证：打开命令提示符（cmd），输入 `java -version`，显示版本1.8即成功。

2.2 Maven (项目构建工具)

1. 下载：<https://maven.apache.org/download.cgi>（选 `apache-maven-3.8.6-bin.zip`），建议使用IDEA自带的Maven，无需安装，跳入第四步去配置即可。
2. 解压到 `D:\apache-maven-3.8.6`（或其他无中文路径）
3. 配置环境变量：

- 新建 `MAVEN_HOME` : `D:\apache-maven-3.8.6`
 - `Path` 添加 `%MAVEN_HOME%\bin`
4. 配置阿里云镜像（加速下载依赖）：
- 打开 `D:\apache-maven-3.8.6\conf\settings.xml`，如果IDEA自带的就选择这个路径：
`{idea安装路径}\plugins\maven\lib\settings.xml`
 - 找到 `<mirrors>` 标签，在里面添加：

```
<mirror>
  <id>aliyunmaven</id>
  <mirrorOf>central</mirrorOf>
  <name>阿里云公共仓库</name>
  <url>https://maven.aliyun.com/repository/public</url>
</mirror>
```

5. 验证：cmd输入 `mvn -v`，显示版本信息即成功。

2.3 MySQL 8.0（数据库）

1. 下载安装包：<https://dev.mysql.com/downloads/mysql/>（选Windows (x86, 64-bit), MSI Installer）
2. 安装时选择“Developer Default”，一路Next，设置root密码为 **123456**（请牢记）。
3. 安装完成后，打开cmd，输入：

```
mysql -u root -p
```

输入密码 `123456`，进入MySQL命令行。

4. 创建项目数据库：

```
CREATE DATABASE car_sales CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

5. 验证：`SHOW DATABASES;` 能看到 `car_sales` 即可。

2.4 Node.js（前端运行环境）

1. 下载LTS版本：<https://nodejs.org/>（如18.x或20.x）。访问 Node.js 官方历史版本发布页：
`https://nodejs.org/dist/`。
2. 安装时保持默认选项，一路Next。
3. 验证：cmd输入 `node -v` 和 `npm -v`，显示版本号即成功。

2.5 Git（版本控制）

1. 下载：<https://git-scm.com/downloads>
2. 安装时保持默认选项，唯一注意：选择默认编辑器为“Use the Nano editor”或“Use Vim”均可。
3. 安装完成后，打开 **Git Bash**（桌面右键菜单或开始菜单），配置用户名和邮箱：

```
git config --global user.name "你的姓名拼音" # 例如 zhangsan
git config --global user.email "你的邮箱@example.com"
```

4. 验证: `git --version`

2.6 Tomcat (Web服务器)

- 1. 下载地址: <https://tomcat.apache.org/download-90.cgi> , 选择 apache-tomcat-9.0.xx-windows-x64.zip。
- 2. 解压到 D:\apache-tomcat-9.0.xx。
- 3. IDEA配置: Run → Edit Configurations → Tomcat Server → Local → 选择解压目录即可。

2.7 IDE (集成开发环境)

- 后端: IntelliJ IDEA 社区版 (下载地址: <https://www.jetbrains.com/idea/download/>)
- 前端: VS Code (下载地址: <https://code.visualstudio.com/>)

三、Git基础入门 (必备技能)

3.1 常用命令速查表

命令	说明
<code>git init</code>	在当前目录初始化Git仓库
<code>git clone <仓库地址></code>	克隆远程仓库到本地
<code>git status</code>	查看当前文件修改状态
<code>git add <文件名></code> 或 <code>git add .</code>	将文件添加到暂存区
<code>git commit -m "提交说明"</code>	将暂存区内容提交到本地仓库
<code>git push origin <分支名></code>	推送本地提交到远程仓库
<code>git pull origin <分支名></code>	拉取远程仓库的最新代码
<code>git branch</code>	查看本地分支
<code>git branch <分支名></code>	创建新分支
<code>git checkout <分支名></code>	切换分支
<code>git checkout -b <分支名></code>	创建并切换到新分支
<code>git merge <分支名></code>	将指定分支合并到当前分支

3.2 团队协作基本流程 (实训使用)

以下以 **Gitee (码云)** 为例，GitHub 操作类似

1、组长操作

第1步：创建远程仓库

1. 登录 Gitee (<https://gitee.com>)，点击右上角 + → **新建仓库**
2. 填写仓库名称（如 `car-sales-group1`），选择 **私有**
3. 点击 **创建**，不要勾选“使用Readme初始化”

第2步：添加组员为开发者

1. 进入仓库 → 管理 → 仓库成员管理 → 邀请用户
2. 搜索组员的 Gitee 账号，添加为 **开发者**

第3步：本地初始化并推送初始代码

```
# 1. 进入项目目录（包含 backend 和 frontend 文件夹）
cd car-sales-group1

# 2. 初始化本地仓库
git init

# 3. 添加所有文件到暂存区
git add .

# 4. 提交到本地仓库
git commit -m "init: 后端脚手架+前端Vue3骨架"

# 5. 关联远程仓库
git remote add origin https://gitee.com/你的用户名/car-sales-group1.git

# 6. 推送到远程（首次推送）
git push -u origin main

# 7. 创建并推送 dev 分支（后续开发使用）
git checkout -b dev
git push origin dev
```

2、组员操作

```
# 1. 克隆仓库到本地
git clone https://gitee.com/组长用户名/car-sales-group1.git

# 2. 进入项目目录
cd car-sales-group1

# 3. 切换到 dev 分支
git checkout dev

# 4. 创建自己的功能分支
git checkout -b feature/自己的功能名 # 例如: feature/backend-car
```

3、每日开发流程

开始工作前（拉取最新代码）

```
git checkout dev
git pull origin dev
git checkout feature/自己的分支名
git merge dev
```

提交代码

```
git add .
git commit -m "feat: 完成xxxxxx"
git push origin feature/自己的分支名
```

每天结束时 (组长合并到 dev)

```
git checkout dev
git pull origin dev
git merge feature/backend-car
git push origin dev
```

注意：不要直接在 `main` 或 `dev` 分支上写代码，一定要建个人功能分支。

四、项目骨架搭建

4.1 后端：导入SSM脚手架

1. 从老师处获取 `backend-sales.zip` (SSM脚手架)，解压到你的工作目录（例如 `D:\workspace`）。
2. 打开IntelliJ IDEA，点击 `File → Open`，选择解压后的 `backend-sales` 文件夹。
3. IDEA会自动下载Maven依赖（第一次较慢，利用阿里云镜像）。右下角进度条走完即可。
4. 修改数据库连接配置：
 - 打开 `src/main/resources/application.properties`
 - 将 `jdbc.password` 改为你安装MySQL时设置的密码（例如 `123456`）
5. 测试连接：
 - 找到老师提供的测试类 `TestDBConnection.java`（一般在 `src/test/java` 下）
 - 右键运行，控制台输出“数据库连接成功”即OK。

脚手架结构简介：

```
backend-sales
├─ pom.xml                # Maven配置文件
├─ src/main/java/com/sales
│   ├─ config            # Spring、SpringMVC、MyBatis配置
│   ├─ controller        # 控制器（空，后续添加）
│   ├─ service           # 业务逻辑（空）
│   ├─ mapper            # 数据访问层（空）
│   ├─ entity            # 实体类（空）
│   ├─ common            # 通用工具类（Result，PageResult）
│   └─ exception         # 全局异常处理
└─ src/main/resources
    ├─ application.properties
    └─ mybatis            # Mapper XML文件（空）
```

4.2 前端：创建Vite + Vue3项目

1. 打开VS Code，打开终端（Terminal → New Terminal）。
2. 进入你希望存放前端代码的目录（例如 `D:\workspace`），执行：

```
npm create vite@4.1.0 frontend-sales -- --template vue
```

- 如果提示 `Need to install the following packages: create-vite@4.1.0`，按 `y` 确认。
3. 进入项目目录并安装依赖：

```
cd frontend-sales
npm install
```

4. 安装额外依赖：

```
npm install vue-router@4.6.4 axios@1.17.0 element-plus@2.14.1 echarts@6.1.0
```

5. 配置Vite代理（解决前后端跨域）：

- 打开 `frontend-sales/vite.config.js`
- 替换全部内容为：

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})
```


6. 封装axios请求库:

- 创建文件夹 `src/Utils`
- 在 `Utils` 下新建 `request.js`, 内容如下:

```
import axios from 'axios'

const request = axios.create({
  baseURL: '/api',
  timeout: 10000
})

// 请求拦截器: 添加token
request.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

// 响应拦截器: 统一处理错误
request.interceptors.response.use(
  response => response.data,
  error => {
    console.error('请求失败', error)
    return Promise.reject(error)
  }
)

export default request
```

7. 配置路由:

- 创建 `src/router/index.js`

```
import { createRouter, createWebHistory } from 'vue-router'

const routes = [
  {
    path: '/',
    component: () => import('../views/Hello.vue') // 稍后创建占位页面
  }
]

const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

8. 创建占位页面:

- 在 `src/views` 下新建 `Home.vue`, 内容:

```
<template>
  <div style="text-align:center; padding:100px; font-size:24px;">
    欢迎使用汽车销售系统
  </div>
</template>
```

9. 修改 `src/main.js`，引入Element Plus和路由：

```
import { createApp } from 'vue'
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import App from './App.vue'
import router from './router'

const app = createApp(App)
app.use(ElementPlus)
app.use(router)
app.mount('#app')
```

10. 修改 `App.vue`：

```
<template>
  <router-view></router-view>
</template>
```

11. 启动前端项目验证：

```
npm run dev
```

浏览器访问 `http://localhost:5173`，能看到“欢迎使用汽车销售系统”即成功。

4.3 创建Git仓库并提交初始代码

1. 在Gitee（或GitHub）创建一个新仓库，命名为 `car-sales-groupX`（X为组号），选择“私有”。
2. 添加所有组员为仓库成员（设置 → 仓库成员管理 → 邀请用户）。
3. 在本地创建一个总目录 `car-sales-group1`，将后端和前端代码放进去。
4. 初始化Git仓库并提交：

```
git init
git add .
git commit -m "init: 后端脚手架+前端Vue3骨架"
git remote add origin https://gitee.com/你的用户名/car-sales-group1.git
git push -u origin main
```

5. 创建 `dev` 分支并推送：

```
git checkout -b dev
git push origin dev
```

6. 每位组员克隆仓库到自己的电脑：

```
git clone https://gitee.com/组长的用户名/car-sales-group1.git
cd car-sales-group1
git checkout dev
git checkout -b feature/自己的分支名 # 例如 feature/backend-car
```

五、常见问题及解决方法

5.1 环境安装类

问题	解决方案
java 不是内部或外部命令	检查 JAVA_HOME 环境变量，Path中是否添加了 %JAVA_HOME%\bin，重启 cmd。
Maven下载依赖很慢	确认 settings.xml 中阿里云镜像配置正确；删除本地仓库（默认 C:\Users\你的用户名\.m2\repository）重新下载。
MySQL连接失败 Access denied	使用 mysql -u root -p 重新输入密码；或重置密码：ALTER USER 'root'@'localhost' IDENTIFIED BY 'newpassword';
npm 命令找不到	重启终端；确认Node.js安装时勾选了“Add to PATH”。

5.2 Git操作类

问题	解决方案
git push 报错 403	使用SSH方式：生成密钥对（ssh-keygen），将公钥添加到 Gitee/GitHub；或使用Personal Access Token代替密码。
git pull 提示冲突	不要慌！先 git stash 暂存本地修改，拉取后 git stash pop，手动解决冲突（老师会演示）。
忘记切换分支，直接在main上写了代码	创建一个新分支：git checkout -b feature/my-work，然后把main 回退到之前的状态。

5.3 项目运行类

问题	解决方案
后端运行测试类报 ClassNotFoundException	Maven依赖未完全下载：在IDEA右侧Maven面板点击“刷新”按钮。
前端 npm run dev 报错 Error: Cannot find module ...	执行 npm install 重新安装依赖。
前端代理不生效，请求404	确认后端已启动（端口8080），vite.config.js 中代理路径配置正确，重启前端服务。

六、今日自检清单

完成以下所有项，才算完成第一天的任务：

- 我所在的小组已确定新主题，并完成了需求分析
- 我能画出新主题的完整业务流程图
- 新主题的6~8张表已完成初步设计，每张表至少有3个核心字段
- 我能说出本组所选主题的三个核心功能
- 我的小组已经完成小组分工，我知道自己负责的模块
- 我的电脑上JDK、Maven、MySQL、Node.js、Git、Tomcat都已安装并能正常运行
- 后端脚手架导入IDEA后，运行测试类显示“数据库连接成功”
- 前端项目执行 `npm run dev` 后，浏览器能打开页面
- 我已经成功将前后端代码提交到Git仓库，并推送到远程
- 我理解了Git的基本命令（add、commit、push、pull、branch、checkout）

七、课后作业

1. **完善新主题需求分析**：各组根据讨论结果，进一步完善所选主题的业务流程图（至少包含3个核心模块）。
2. **完成新主题表结构设计**：根据业务流程图，设计6~10张表的完整SQL建表语句（含字段注释、主外键关系），参考老师提供的 `car_sales.sql` 案例，提交到Git仓库。
3. **复习Git**：独立完成一个练习：创建仓库、添加文件、提交、推送到远程、创建分支、合并。
4. **环境准备**：确保明天上课前，你的后端和前端项目都能顺利运行。

遇到无法解决的问题，第一时间在小组内讨论，或通过QQ/微信联系老师。